1. Виконавець: Демчук Дмитро, група К-25. Лабораторна робота №4, варіант №13.

2. Умова (частина, специфічна для варіанта): структура з двома цілими полями (m = 2). Частоти операцій: поле 0 - read 10%, write 10%; поле 1 - read 50%, write 10%; запит рядка (string) - 20%.

3. Схема захисту даних: два `std::mutex`, по одному на кожне поле (перший захищає значення поля 0, другий - поля 1). Операції read і write блокують лише м'ютекс свого поля, тому операції над різними полями не заважають одна одній. Запит рядка (20%) бере обидва м'ютекси через `std::scoped_lock`, що виключає взаємоблокування, та лише копіює два числа; сам рядок формується вже поза блокуванням, тож критична секція найкоротша. Кожне поле разом зі своїм м'ютексом лежить в окремому 64-байтному рядку кешу (`alignas(64)`), щоб запис в одне поле не гальмував роботу з іншим. Атомарні змінні та контейнери не використано, лише звичайний масив розміру m. `std::shared_mutex` перевірено окремо у пробному замірі на Linux (читання складають 70% операцій), але він виявився повільнішим: наприклад, для файлу за умовою на 3 потоках 582 мс проти 135 мс з `std::mutex`. Критичні секції надто короткі, а кожне читання під `shared_mutex` теж змінює спільну пам'ять. Тому обрано `std::mutex`.

4. Усереднений час виконання, мс (10 запусків; у кожній клітинці виконується однакова кількість операцій, 3 000 000, які діляться між потоками; час зчитування файлу не враховано):

Файл	**1 потік**	**2 потоки**	**3 потоки**
За умовою (expected)	51,24	88,83	136,49
Рівні частоти (even)	59,34	93,43	157,42
96% string (mostly_string)	138,69	244,61	384,48

5. Висновки: таблиця не відповідає наївному очікуванню прискорення. На 2 потоках час зростає у 1,6-1,8 раза порівняно з одним, на 3 потоках - у 2,6-2,8 раза. Ймовірна причина: самі операції дуже короткі (десятки наносекунд), тому практично весь час іде на синхронізацію, а розпаралелювати нічого, до того ж більшість операцій (60%) стосується одного й того самого поля 1 і одного м'ютекса, за який потоки конкурують; передача рядка кешу між ядрами коштує дорожче за саму операцію. Найповільнішим є файл з переважанням string: формування рядка потребує виділення пам'яті, а кожен запит бере обидва м'ютекси. З одним потоком конкуренції немає, тому він найшвидший. Для коротких операцій над спільною структурою додаткові потоки прискорення не дають; їх виправдовує лише довша робота поза критичною секцією.

6. Виконано самостійно: розроблено структуру Fields (м'ютекс на поле, `operator std::string`, вирівнювання по рядках кешу); програму вимірювання, яка завантажує файли до запуску таймера, ділить однакову загальну кількість операцій між 1, 2 і 3 потоками й усереднює результати; перевірку вхідних файлів з повідомленнями про помилки; генератор трьох типів тестових файлів (за умовою, рівні частоти, 96% string); порівняння `std::mutex` і `std::shared_mutex`; перевірку на гонки даних (ThreadSanitizer) та збирання через CMake в режимі Release.